This problem set is not going to be collected, although solutions will be posted before the quiz. Unlike a problem set collected for credit, though, you are free to discuss with classmates and course staff and get help. I recommend you first attempt the problems on your own, then discuss with classmates. After that, you may want to consult course staff at help hours or on Ed Discussion. After solutions are posted, consult the solutions and compare to what you wrote, and then discuss the differences with your study group.

1. I am teaching a class that has n teaching assistants, each of whom hold office hours outside, at JavaCity. The ith TA holds office hours starting at time S_i and ending at time E_i. Each TA holds exactly one office hour (which may last any amount of time, not necessarily one hour, and different TAs may have different interval lengths, which may overlap with one another). Attendance has been low because students, being ICS majors, are afraid of the sun, so the department announces that all TAs who do not have a student visit them this week will be fired[1]. Because all of our teaching assistants are awesome[2], we want to ensure that none get fired. However, due to student fears of the sun, we want to minimize the total number of visits made to TAs.

   For purposes of this problem, assume each TA's office hour time is one continuous interval with no breaks, and that a student visiting the sundeck at time t counts as visiting all TAs whose office hours interval contains t. Also assume that student visits are "instantaneous," in the sense that the amount of time a student stays in the sun is negligible – formally, each student visits the deck at a single point in time. This also helps to avoid exposing a student to the sun unnecessarily.

   You propose the following: first, we sort all of the intervals by end time. We send a student to attend office hours at the moment immediately prior to the end of the first ending interval. We remove from our input all TAs who overlap with this time, and if the remaining set is non-empty, we repeat.

   Prove that the **greedy algorithm** in the previous statement minimizes the number of students we need to send to visit TAs. For purposes of this homework this quarter, it is sufficient to demonstrate that for any optimal solution, there is an equally optimal choice of student visits that includes your first choice.

2. We'd like to select frequencies for FM radio stations so that no two are too close in frequency (creating interference). Suppose there are n candidate frequencies $\{f_1 f_2 \dots f_n\}$. Our goal is to pick as many frequencies as possible such that no two selected frequencies f_i, f_j have $|f_i - f_j| < \varepsilon$ (for a given input variable ε). Give an efficient greedy algorithm to solve this problem and prove that it is correct.

3. Consider a currency which has 1 cent, 7 cent, 35 cent, and 140 cent coins. Give a greedy algorithm that, given integer input n, outputs a number of coins of each denomination so that the coins sum to n cents, using the smallest total number of coins. What is the property of these numbers which makes your algorithm work? Give a set of denominations of coins for which your algorithm (generalized to those denominations) will not work.

[1] This part is not actually true and is fiction for the problem.
[2] Worth noting: this part is true for CompSci 161 this quarter.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

4. The MAXIMUM INDEPENDENT SET problem is as follows. Given an undirected graph $G = (V, E)$, a subset $I \subseteq V$ of the vertices is called an *independent set* if no two vertices in set I share an edge. One way to picture this is vertices are people, edges represent conflicts, and an independent set is a party[3] that doesn't cause an argument or a fight. In general, finding the maximum independent set in a graph is a difficult problem. However, suppose our input graph *is a tree*. Give an efficient algorithm to compute the maximum independent set in a tree. Justify the correctness of your algorithm.

5. We're asked to help the captain of the UCI's tennis team to arrange a series of matches against Long Beach State's team[4]. Both teams have n players; the tennis rating (a positive number, where a higher number can be interpreted to mean a better player) of the ith member of UCI's team is a_i and the tennis rating for the kth member of LBSU's team is s_k. We would like to set up a competition in which each person plays one match against a player from the opposite school. Because we get to select who plays against whom, our goal is to make sure that in *as many matches as possible*, the UCI player has a higher tennis rating than their opponent.

   (a) One proposal has been to sort both rosters by tennis rating, and then pair our best player against their best player, our second best against their second best, and so on in that fashion. Give a complete counter-example to demonstrate that this does not solve the problem stated here.

   (b) Give an efficient **greedy** algorithm for this problem and prove that it is correct (maximizes the number of matches in which our team has the player with the higher tennis rating).

   For CompSci 161 this quarter, for the proof, it is sufficient to demonstrate that the repair of an arbitrary inversion in any alternate ordering, compared to yours, produces an ordering that is no worse. You may use the fact from lecture that any permutation that differs from yours has a consecutive pair of elements that are inverted relative to yours, however for this particular problem, the proof may not need the fact that they are adjacent.

6. Suppose we have a set of n homework assignments, each of which have the **same deadline**. However, they do not all take the same amount of time to do: assignment i takes $t_i > 0$ minutes to do. For each assignment turned in prior to the deadline, we get zero penalty. For each assignment turned in after the deadline, we lose one point for each minute it is late. For example, if two homework assignments are late, one by one minute and the other by three minutes, we lose four points.

   Our goal is to minimize the number of points we lose. Prove that doing our homework assignments in order of *shortest task first* (i.e., non-decreasing value of t_i; we do the shortest task, then the second shortest, and so on) will optimize this objective.

[3] It's always a party in CompSci 161

[4] Not really.

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.

7. The following is an abstraction of data caching, which you may have learned about in ICS 51 and 53. However, you do not need knowledge from those classes for this problem. The particular description of the problem comes from *Algorithm Design* by Kleinberg and Tardos.

   Consider a set U of n pieces of data stored in *main memory*. We also have a faster memory, the cache, that can hold $k < n$ pieces of data at any one time. We will assume that the cache initially holds some set of k items. A sequence of data items $D = d_1, d_2, \ldots d_m$ drawn from U is presented to us : this is the sequence of memory references we must process – and in processing them we must decide at all times which k items to keep in the cache. When item d_i is presented, we can access it very quickly if it is already in the cache; otherwise, we are required to bring it from main memory into the cache and, if the cache is full, to *evict* some other piece of data that is currently in the cache to make room for d_i. This is called a *cache miss* and we want to have as few of these as possible.

   Thus, on a particular sequence of memory references, a cache maintenance algorithm determines an *eviction schedule* – specifying which items should be evicted from the cache at which points in the sequence – and this determines the contents of the cache and the number of misses over time.

   Example: if cache size $k = 2$ and the requests are a, b, c, b, c, a, b, we have items a, b to start in cache, then at the third request, we evict a for c, then at the sixth request, we evict c for a. Thus we have two misses for this input.

   Show that the policy of evicting whichever item in the cache will be next requested *farthest in the future* (among those in the cache currently) will minimize evictions. Obviously a real cache couldn't implement this, but that's not the point of this exercise.

   (a) We call an eviction schedule *reduced* if the only time it performs evictions is when a data element is requested, but not already in the cache. This means, for example, that it doesn't preemptively evict a data element. Prove that for any (possibly not reduced) schedule S, there is a reduced schedule S' that has no more evictions.
   This means we don't have to investigate any algorithms for this problem that do anything other than evictions made to bring into the cache the currently-requested data element.

   (b) Let S_{FF} be the schedule produced by the farthest in future algorithm described in the problem statement and let OPT be a schedule has the fewest cache misses. Show that we can convert any OPT into S_{FF} without increasing the number of cache misses. *This demonstrates that no schedule can have fewer misses than S_{FF}, because we can convert any optimal into S_{FF} without making it worse.*

This homework (approximately) covers Chapter 10 in the textbook of Goodrich and Tamassia. One of the strengths of this book is that it has a good variety and quality of practice problems. If you need help deciding which problems to do, consider trying R-10.4, R-10.5, R-10.6, R-10.7, R-10.8, R-10.9, R-10.10, R-10.4, R-10.5, R-10.6, R-10.7, R-10.8, A-10.4, A-10.5, A-10.7

I also suggest the following problems from the Erickson textbook; within Chapter 4, problems 1, 3, 4, 5, 14(a), 14(b), 16, 17, 20(a), 20(b), 21(a), 21(b), 24.

For additional reinforcement with the topic of Huffman Trees, consider problem 6

© CompSci 161 Fall 2025– Michael Shindler– University of California, Irvine
This Problem Set may not be reposted without the express written permission of the professor teaching this course.